

CS406 Digital Image Processing

Homework 2

Department of Computer Science
University of Management and Technology
Summer 2026

Due: August 22, 2026

Instructions

- Total Questions: 5
- Each question has a **numerical problem** and a **Python programming** component.
- Submit a single PDF report with your answers, code, and output screenshots.
- Submit your Python scripts as separate .py files.
- Use Python libraries: NumPy, OpenCV, scikit-image, Matplotlib.

1 Question 1: Image Enhancement and Histogram Equalization

Part A: Numerical Problem

The histogram of an 8-bit grayscale image is given below:

Intensity r_k	0	1	2	3	4	5	6	7
Frequency n_k	50	100	150	200	250	150	70	30

Table 1: Histogram of the image (total pixels $N = 1000$)

- Compute the probability density function (PDF) $p(r_k) = n_k/N$ for each intensity level.
- Compute the cumulative distribution function (CDF) $c(r_k) = \sum_{j=0}^k p(r_j)$.
- Using histogram equalization, compute the mapping $s_k = \text{round}((L - 1) \cdot c(r_k))$ where $L = 8$.
- Determine the equalized histogram frequencies for each output intensity s_k .

Part B: Programming Problem

Write a Python program to:

- Load a grayscale image of your choice.
- Compute and plot the original histogram.
- Apply histogram equalization using:
 - Manual implementation (using NumPy only, no built-in equalization functions).
 - Built-in function from OpenCV: `cv2.equalizeHist()`.
- Display:
 - Original and equalized images side-by-side.
 - Histograms of original and equalized images.
- Compare the results and discuss the improvement in contrast.

2 Question 2: Geometric and Affine Transformations

Part A: Numerical Problem

A point $(x, y) = (3, 2)$ undergoes the following transformations in order:

1. Rotation by $\theta = 45^\circ$ counterclockwise about the origin.
 2. Scaling by factors $s_x = 2.0$ and $s_y = 1.5$.
 3. Translation by $(t_x, t_y) = (10, 5)$.
- (a) Write the homogeneous transformation matrix for each transformation.
- (b) Compute the combined transformation matrix $\mathbf{M} = \mathbf{T} \cdot \mathbf{S} \cdot \mathbf{R}$.
- (c) Compute the final transformed coordinates (x', y') using $\mathbf{p}' = \mathbf{M} \cdot \mathbf{p}$.
- (d) Verify your result by applying the transformations sequentially.

Part B: Programming Problem

Write a Python program to:

1. Load a color image.
2. Apply the following affine transformations using OpenCV:
 - Translation by $(50, 30)$ pixels.
 - Rotation by 30° about the image center.
 - Scaling by $1.5\times$.
 - Shear transformation.
 - Custom affine transformation (combination of rotation, scaling, translation).
3. For each transformation:
 - Display the transformed image.
 - Discuss the effect on the image.
 - Explain how the transformation matrix was constructed.
4. Implement inverse warping (backward mapping) for rotation and display the result.

3 Question 3: Spatial Filtering

Part A: Numerical Problem

Consider a 5×5 grayscale image patch:

$$\begin{bmatrix} 10 & 12 & 15 & 18 & 20 \\ 12 & 14 & 16 & 20 & 22 \\ 15 & 16 & 18 & 22 & 25 \\ 18 & 20 & 22 & 24 & 26 \\ 20 & 22 & 25 & 26 & 28 \end{bmatrix}$$

- (a) Apply a 3×3 mean filter (box blur) to compute the output value at the center pixel (2, 2) (0-indexed).
- (b) Apply a 3×3 median filter to compute the output value at the center pixel (2, 2).
- (c) Apply a 3×3 Laplacian filter:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

to compute the output at the center pixel.

- (d) Apply a 3×3 Sobel operator (horizontal and vertical) to compute the gradient magnitude at the center pixel.

Part B: Programming Problem

Write a Python program to:

1. Load a grayscale image (preferably with noise).
2. Implement the following spatial filters manually (using convolution with NumPy, not built-in functions):
 - Mean filter (average blur) with kernel sizes 3×3 , 5×5 , 7×7 .
 - Median filter with kernel sizes 3×3 , 5×5 .
 - Gaussian filter with $\sigma = 1.0$, kernel size 5×5 .
 - Laplacian filter for edge detection.
 - Sobel operator for gradient magnitude.
3. Compare your manual implementation with OpenCV built-in functions.
4. Display:
 - Original image.
 - Filtered images for each filter.
 - Comparison of different kernel sizes.
5. Discuss the effect of each filter and the trade-offs between different kernel sizes.

4 Question 4: Image Blending and Compositing

Part A: Numerical Problem

Given the following pixel values in normalized format $[0, 1]$:

- Foreground color: $(R_F, G_F, B_F) = (0.8, 0.2, 0.4)$ with alpha $\alpha_F = 0.7$
 - Background color: $(R_B, G_B, B_B) = (0.1, 0.5, 0.3)$ with alpha $\alpha_B = 1.0$
- (a) Compute the blended color using the simple alpha blending formula: $C = (1 - \alpha_F) \cdot B + \alpha_F \cdot F$.
- (b) Using the alpha compositing formulas:
- (a) Normalize alpha values.
 - (b) Compute the composite alpha $\alpha_C = \alpha_F + \alpha_B(1 - \alpha_F)$.
 - (c) Compute the blended color $C = \frac{F \cdot \alpha_F + B \cdot \alpha_B(1 - \alpha_F)}{\alpha_C}$.
- (c) Compare the results from (a) and (b) and explain the difference.
- (d) What would be the result if $\alpha_F = 0$ or $\alpha_F = 1$?

Part B: Programming Problem

Write a Python program to:

1. Load two images (foreground and background) of your choice.
2. Resize images to the same dimensions if needed.
3. Perform the following blending operations:
 - Linear alpha blending with varying α values ($\alpha = 0.0, 0.3, 0.5, 0.7, 1.0$).
 - Alpha compositing with proper alpha handling.
 - Overlay blending.
 - Hard Light blending.
 - Soft Light blending.
4. Implement the following from scratch (using NumPy):
 - Multiply blend mode.
 - Screen blend mode.
 - Overlay blend mode.
5. Display:
 - Foreground, background, and blended images.
 - Comparison of different blend modes.
 - Comparison of simple alpha blending vs. alpha compositing.
6. Discuss the applications of image blending in computer graphics and photo editing.

5 Question 5: Frequency Domain Filtering

Part A: Numerical Problem

Given a 4×4 image:

$$f(x, y) = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

- (a) Compute the 2D Discrete Fourier Transform (DFT) $F(u, v)$ manually (using the DFT formula).
- (b) Apply a low-pass filter in the frequency domain by zeroing out all frequencies except the DC component and its immediate neighbors (a 2×2 block in the center of the frequency domain).
- (c) Compute the inverse DFT to obtain the filtered image.
- (d) Apply a high-pass filter by keeping only the high-frequency components and zeroing out the low frequencies.
- (e) Compare the results of low-pass and high-pass filtering in terms of the resulting image.

Part B: Programming Problem

Write a Python program to:

1. Load a grayscale image.
2. Perform frequency domain filtering using:
 - Compute the 2D FFT using `np.fft.fft2()`.
 - Shift the zero-frequency component to the center using `np.fft.fftshift()`.
 - Create the following filters in the frequency domain:
 - Ideal Low-Pass Filter (ILPF) with different cutoff frequencies.
 - Ideal High-Pass Filter (IHPF) with different cutoff frequencies.
 - Gaussian Low-Pass Filter (GLPF) with different σ values.
 - Gaussian High-Pass Filter (GHPF) with different σ values.
 - Butterworth Low-Pass Filter (BLPF) of order n .
 - Apply the filters by multiplying with the frequency spectrum.
 - Shift back and compute the inverse FFT to obtain the filtered image.
3. Display:
 - Original image.
 - Frequency spectrum (magnitude) as an image.
 - Filtered images for each filter type.

- Comparison of different cutoff frequencies and filter orders.

4. Discuss:

- The effects of low-pass and high-pass filtering.
- The advantages and disadvantages of ideal vs. Gaussian vs. Butterworth filters.
- How frequency domain filtering compares to spatial domain filtering.
- Applications of frequency domain filtering in image processing.

Appendix: Useful Python Code Snippets

Loading and Displaying Images

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Load image
6 img = cv2.imread('image.jpg') # BGR format
7 img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # Convert to RGB
8 img_gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
9
10 # Display
11 plt.figure(figsize=(10,5))
12 plt.subplot(1,2,1), plt.imshow(img_rgb), plt.title('Original RGB')
13 plt.subplot(1,2,2), plt.imshow(img_gray, cmap='gray'), plt.title('
    Grayscale')
14 plt.show()
```

Histogram Equalization

```
1 # Manual histogram equalization
2 def hist_equalize(img):
3     hist, bins = np.histogram(img.flatten(), 256, [0, 256])
4     cdf = hist.cumsum()
5     cdf_normalized = cdf * 255 / cdf[-1]
6     img_eq = np.interp(img.flatten(), bins[:-1], cdf_normalized)
7     return img_eq.reshape(img.shape).astype(np.uint8)
```

Affine Transformation

```
1 # Translation
2 M_trans = np.float32([[1, 0, 50], [0, 1, 30]])
3 img_trans = cv2.warpAffine(img, M_trans, (img.shape[1], img.shape[0]))
4
5 # Rotation
6 M_rot = cv2.getRotationMatrix2D((img.shape[1]/2, img.shape[0]/2), 30,
    1)
7 img_rot = cv2.warpAffine(img, M_rot, (img.shape[1], img.shape[0]))
```

Spatial Filtering

```
1 # Manual convolution
2 def convolution2d(image, kernel):
3     k_h, k_w = kernel.shape
4     h, w = image.shape
5     output = np.zeros((h, w))
6     pad_h, pad_w = k_h // 2, k_w // 2
7     padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='
    constant')
8
9     for i in range(h):
```

```

10         for j in range(w):
11             output[i, j] = np.sum(padded[i:i+k_h, j:j+k_w] * kernel)
12     return output

```

Frequency Domain Filtering

```

1  # FFT and filtering
2  f = np.fft.fft2(img_gray)
3  f_shift = np.fft.fftshift(f)
4  magnitude_spectrum = 20 * np.log(np.abs(f_shift) + 1)
5
6  # Create Gaussian Low-Pass Filter
7  rows, cols = img_gray.shape
8  crow, ccol = rows//2, cols//2
9  D0 = 30 # cutoff frequency
10
11 glpf = np.zeros((rows, cols))
12 for i in range(rows):
13     for j in range(cols):
14         D = np.sqrt((i-crow)**2 + (j-ccol)**2)
15         glpf[i, j] = np.exp(-(D**2) / (2 * D0**2))
16
17 # Apply filter
18 f_filtered = f_shift * glpf
19 f_inv_shift = np.fft.ifftshift(f_filtered)
20 img_filtered = np.fft.ifft2(f_inv_shift).real
21 img_filtered = np.clip(img_filtered, 0, 255).astype(np.uint8)

```

Submission Guidelines

- **Part A (Numerical):** Show all steps clearly. Include formulas used and intermediate calculations.
- **Part B (Programming):**
 - Include well-commented code.
 - Include screenshots of output.
 - Discuss the results.
- **Total Marks:** 100 (5 questions $\times$ 20)

Format:

Single PDF file + individual .py files for each question.

- **Deadline:** August 22, 2026, 11:59 PM.